MINI PROJECT REPORT

ON

RESTAURANT RESERVATION SYSTEM

A web-based platform for restaurant discovery, table reservations, and administrative monitoring

---

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

BACHELOR OF ENGINEERING

IN ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SUBMITTED BY

Team Member	Roll Number
Vansh Sharma	2401109
Hammad Shaikh	2401106
Vinayak Kumar Singh	2401117

SUBJECT: ASD&D

PROJECT GUIDE: Prof. Sanobar Shaikh

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

THADOMAL SHAHANI ENGINEERING COLLEGE

Advocate Nari Gursahani Marg, 37th Road, Off Linking Road, Bandra (West), Mumbai - 400050

UNIVERSITY OF MUMBAI | ACADEMIC YEAR 2026-27

THADOMAL SHAHANI ENGINEERING COLLEGE

Department of Artificial Intelligence & Data Science

University of Mumbai | Academic Year 2026-27

CERTIFICATE

This is to certify that the mini project report entitled **"Restaurant Reservation System"** is a record of the project work carried out by the following students: Vansh Sharma (2401109), Hammad Shaikh (2401106), and Vinayak Kumar Singh (2401117), in partial fulfilment of the requirements for the degree of Bachelor of Engineering in Artificial Intelligence and Data Science under the University of Mumbai.

The report presents the observed application screens and project workflow evidence supplied by the team, including restaurant discovery, reservation entry, administrative activity logs, work tracking, build status, and system monitoring.

PROJECT GUIDE

_____

Prof. Sanobar Shaikh

_____ _____

Head of Department **External Examiner**

Abstract

The Restaurant Reservation System is a browser-based application intended to make restaurant discovery and table booking more convenient while giving administrators a consolidated view of system activity. The user-facing interface includes Login and Register options, restaurant search by name, cuisine, or location, and a reservation form that collects the selected restaurant, date, time, number of guests, and contact number. A “My Reservations” entry is also visible in the booking area.

The administrative view provides summary cards for registered users, restaurants, and bookings across time periods, along with a system activity log. The supplied screenshots show supporting engineering workflows around the application: Jira is used to track sprint work, Jenkins displays build history, Prometheus reports the health of monitored targets, and Grafana visualizes host and service resource indicators. A Node Exporter endpoint is also shown as part of the monitoring setup.

The screenshots represent a point-in-time view of the application and its development environment. They demonstrate the presence of the listed interfaces and monitoring surfaces, but do not independently prove every end-to-end behavior, reservation concurrency guarantee, or production performance characteristic. Such behavior should be confirmed through functional, integration, security, and load testing.

Keywords: Restaurant reservation, restaurant search, user authentication, admin dashboard, activity logs, Jira, Jenkins, Prometheus, Grafana, Node Exporter.

Report Overview

Chapter 1 introduces the problem, objectives, and scope. Chapter 2 describes the visible system modules and supporting engineering tools. Chapter 3 summarizes the evidence captured in the screenshots and discusses limitations. The final sections present the conclusion, future work, and references.

Table of Contents

Section	Page
Certificate	2
Abstract	3
Table of Contents	4
List of Figures	5
1. Introduction	6
1.1 Background	6
1.2 Problem Statement	6
1.3 Objectives	6
1.4 Scope	6
2. System Overview and Implementation	7
2.1 User-facing Modules	7
2.2 Administration and Activity Logging	7
2.3 Module Summary	7
2.4 Engineering Workflow and Monitoring	8
3. Results and Discussion	9
3.1 Application Results	9
3.2 CI/CD and Monitoring Results	9
3.3 Limitations and Discussion	9
Screenshots and Evidence	10-15
Conclusion and Future Scope	16
References	17

List of Figures

Figure	Page
1. Login screen	10
2. Admin dashboard summary	10
3. System activity logs	11
4. Reservation form	11
5. Restaurant search result	12
6. Jenkins pipeline status	12
7. Jenkins change history	13
8. Grafana resource monitoring dashboard	13
9. Jira backlog and sprint planning	14
10. Jira board statuses	14
11. Prometheus target health	15
12. Node Exporter landing page	15

Chapter 1 - Introduction

1.1 Background

Restaurant bookings are often handled through calls, messaging, or separate reservation channels. These approaches can make it difficult for customers to compare available restaurants and enter booking details in a consistent way. Administrators also need a way to review basic usage statistics and inspect important system events without manually combining records from multiple places.

The Restaurant Reservation System brings restaurant discovery, reservation entry, and administrative visibility into a single web interface. The submitted screens show a restaurant listing with cuisine, location, seat capacity, and rating, plus a reservation form and a dashboard for high-level counts and event history. The interface uses a warm, restaurant-oriented visual theme with a clear separation between public-facing and administrative screens.

1.2 Problem Statement

The project addresses the need for a simple digital workflow through which customers can find a restaurant and submit reservation details, while administrators can review summarized activity and recent events. A further project objective is to make development and operation more observable through work tracking, automated build status, and resource-monitoring tools.

1.3 Objectives

1. Provide login and registration entry points for users.
2. Allow restaurant discovery through a search field for name, cuisine, or location.
3. Capture the restaurant, date, time, party size, and contact number in a reservation form.
4. Provide a “My Reservations” entry point and an administrative summary of booking activity.
5. Record system activity in an audit-style log that identifies event time, action, entity, user, and details.
6. Track development work in Jira, inspect build execution in Jenkins, and expose monitoring data through Prometheus and Grafana.

1.4 Scope

The report covers only capabilities and tool states visible in the provided screenshots. It does not assume online payment processing, external restaurant-system integration, SMS/email notifications, or any particular frontend framework or database because those details were not supplied. The screenshots are treated as implementation evidence at the time they were captured, not as a substitute for a complete test report.

Chapter 2 - System Overview and Implementation

2.1 User-facing Modules

Authentication interface. The login view contains an email field, password field, a Sign In action, and a Register option. A tab switch allows the user to move between Login and Register. The supplied screenshot demonstrates the interface; authentication security and registration validation should be verified with dedicated tests.

Restaurant discovery. The Restaurants page includes a search field with the prompt “Search by name, cuisine, or location” and a Search action. The captured result lists Kubera Veg restaurant with the category “Indian Chinese,” location “Vasai,” capacity of 20 seats, and a displayed rating of 4.00. A Book action is available beside the listing.

Reservation entry. The booking form collects the restaurant, date, time, number of guests, and contact number. The number of guests is shown with a default value of 1 in the captured screen. The form includes a Confirm Reservation action and the booking area also presents a My Reservations control. A complete acceptance test should confirm required-field validation, booking persistence, duplicate/conflicting reservation handling, and the behavior after confirmation.

2.2 Administration and Activity Logging

The Admin Dashboard is labelled as read-only monitoring. The captured summary cards show 7 registered users, 1 restaurant, 0 bookings today, 0 bookings this week, 2 bookings this month, and 0 upcoming seats booked. These figures are sample values from the screenshot and should not be treated as permanent system totals.

The System Activity Logs section reports 9 total events in the captured view. Its table contains Time, Action, Entity, User, and Details columns. Visible examples include user registration events and reservation creation events. This structure can help administrators trace activity; future improvements should include pagination, filtering, structured detail expansion, and access restrictions for potentially sensitive fields such as email addresses.

2.3 Module Summary

Module	Visible responsibility
Authentication	Login and registration entry points.
Restaurant discovery	Search by name, cuisine, or location; display basic restaurant information.
Reservation	Collect date, time, guest count, and contact number for a booking.
Admin dashboard	Summarize user, restaurant, and booking counts.
Activity log	List system events with timestamps and user/entity context.

Chapter 2 - Engineering Workflow and Monitoring

2.4 Work Tracking with Jira

The Jira screenshots show a project space titled “restaurant reservation.” The sprint is named “Restaurant Reservation Sprint” and includes RR-9 User registration and login, RR-10 Browse and search restaurants, RR-11 Check real-time seat availability, and RR-12 Book restaurant seats. In the captured board, RR-9 is To Do, RR-10 is In Progress, and RR-11 and RR-12 are Done. The backlog also lists work items for Docker environment setup, Jenkins CI/CD, Prometheus and Grafana monitoring, and Jira/GitHub integration.

2.5 Build and Delivery Workflow with Jenkins

The Jenkins page shows the restaurant-reservation pipeline and identifies build #5 as the last, stable, successful, and completed build, while build #4 is shown as the last failed/unsuccessful build. The change history lists work such as configuring the Jenkins Docker pipeline, adding Prometheus and Grafana monitoring, adding a local deployment stage, and fixing local Docker deployment. This is evidence of an iterative build workflow; it does not on its own confirm that every deployment target or release path is production-ready.

2.6 Monitoring with Prometheus, Node Exporter, and Grafana

The Prometheus target-health screenshot lists three target groups: backend, node_exporter, and prometheus. Each is shown as 1/1 up. The backend target exposes a /metrics endpoint on port 5001, Node Exporter is shown on port 9100, and Prometheus is shown on port 9090. These are the endpoints visible in the captured local environment.

The Grafana dashboard is titled “Restaurant Reservation Monitoring” and includes panels for CPU usage, memory usage, disk usage, and network traffic. The captured gauges display 72.6 for CPU usage, 19.5 for memory usage, and 12.6% for disk usage. The unit labels for the first two values are not clear in the screenshot, so they should not be interpreted as percentages without checking dashboard panel configuration. Node Exporter’s landing page includes a Metrics link and links to heap and CPU profile downloads.

2.7 Components Evidenced by the Screenshots

Component	Evidence in the supplied captures
Web application	Login/register, restaurant search, reservation form, admin summary, activity logs.
Jira	Sprint and backlog work items with status columns.
Jenkins	Pipeline status and recent build/change history.
Prometheus	Health page lists backend, Node Exporter, and Prometheus targets as UP.
Grafana	Dashboard visualizes CPU, memory, disk, and network panels.
Node Exporter	Landing page and resource metrics endpoint.

Chapter 3 - Results and Discussion

3.1 Application Results

The submitted screenshots demonstrate the following visible results: (1) the application presents a login and registration interface; (2) users can search a restaurant catalogue and see summary information for a result; (3) a reservation form is available with the core booking fields; and (4) an administrator can view usage summary cards and system activity records. The reservation and activity screens are consistent with the project's goal of bringing discovery, booking entry, and operational visibility into one interface.

The particular restaurant shown in the captured listing is Kubera Veg restaurant in Vasai, with Indian Chinese cuisine, a displayed capacity of 20 seats, and a rating of 4.00. The admin dashboard snapshot shows 7 registered users, 1 restaurant, and 2 bookings this month. The activity log snapshot reports 9 total events. All of these values are recorded here solely to describe the screenshots; they may change as data changes.

3.2 CI/CD and Monitoring Results

Jenkins build #5 is marked successful in the supplied pipeline view, while build #4 is marked failed. The corresponding change list documents ongoing work on Docker deployment and monitoring. Prometheus reports all three displayed targets as UP at the moment of capture. Grafana displays resource panels across a "Last 6 hours" time range. Together, these screenshots provide evidence that a basic development workflow and a monitoring view have been configured.

3.3 Discussion and Limitations

The project combines customer-facing booking screens with administrative reporting and supporting development/operations tools. Jira provides traceability for implementation work, Jenkins records build outcomes, and Prometheus/Grafana provide a place to inspect service and host metrics. These tools improve visibility but do not, by themselves, establish correctness, scalability, or security.

The screenshots do not show a successful reservation confirmation, a populated My Reservations page, automated notification delivery, concurrency tests, or load-test results. Accordingly, this report does not claim those outcomes. Before production use, the team should validate required-field handling, available-seat calculations, simultaneous bookings, authorization boundaries, password handling, audit-log privacy, database recovery, and alert behavior. The Grafana dashboard should also be reviewed for correct units and metric labels; the captured network panel appears to repeat the "Network Receive" legend.

3.4 Suggested Test Checklist

- Verify valid and invalid login/register flows and role-based access to the admin dashboard.
- Test restaurant search with exact names, cuisine keywords, locations, empty input, and no-match input.
- Test reservations with missing fields, past dates, guest counts above capacity, and overlapping bookings.
- Confirm activity records are created correctly and sensitive details are not exposed to unauthorized users.
- Exercise application failure and recovery while observing Prometheus target state and Grafana panels.

Screenshots and Evidence - Application Entry and Administration

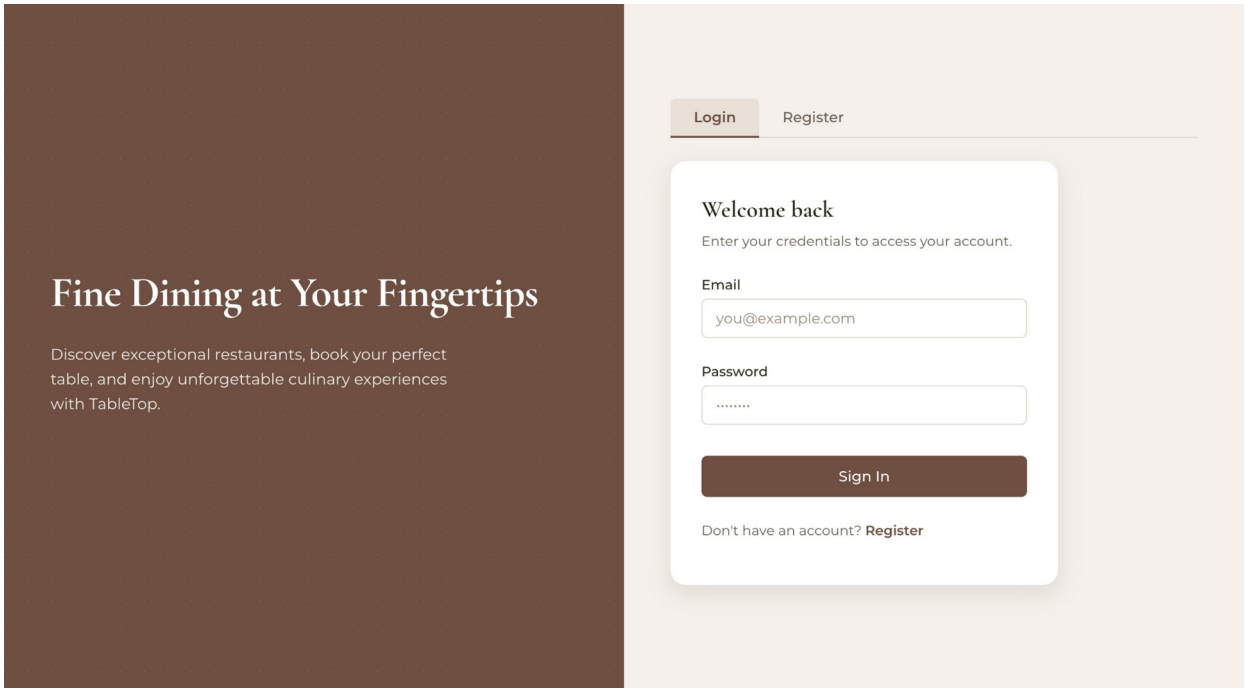


Figure 1. Login screen with Login and Register tabs.

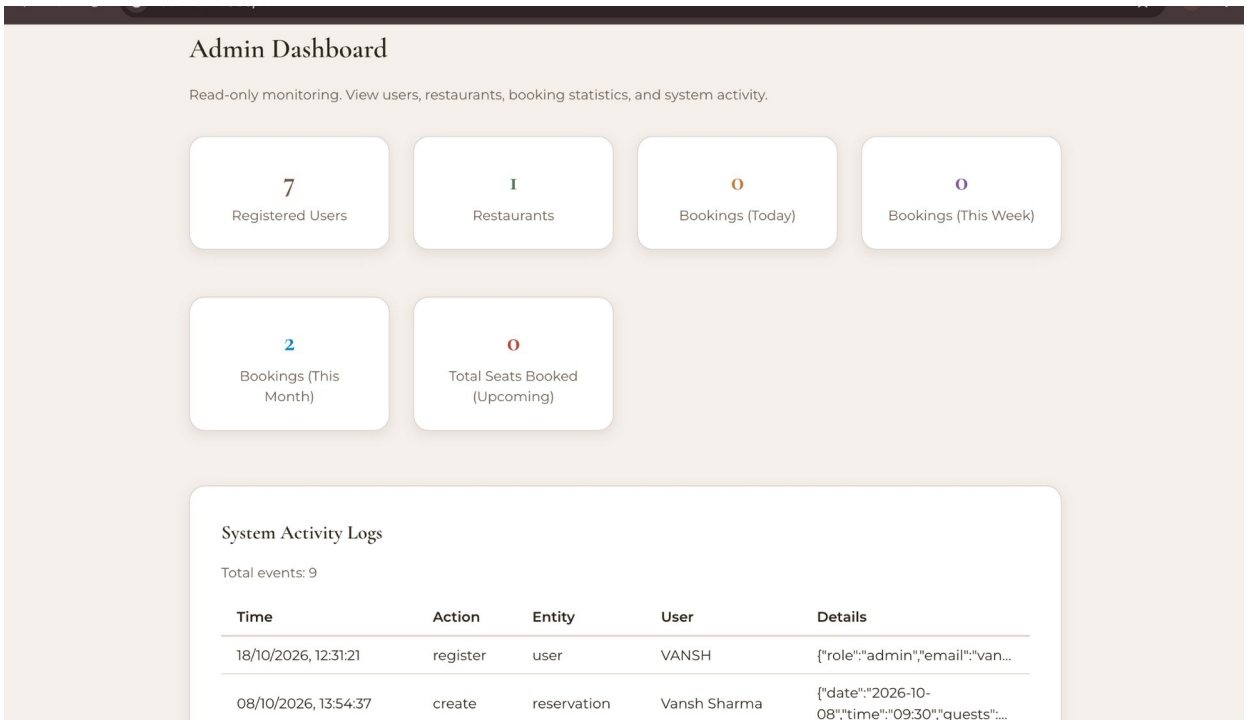
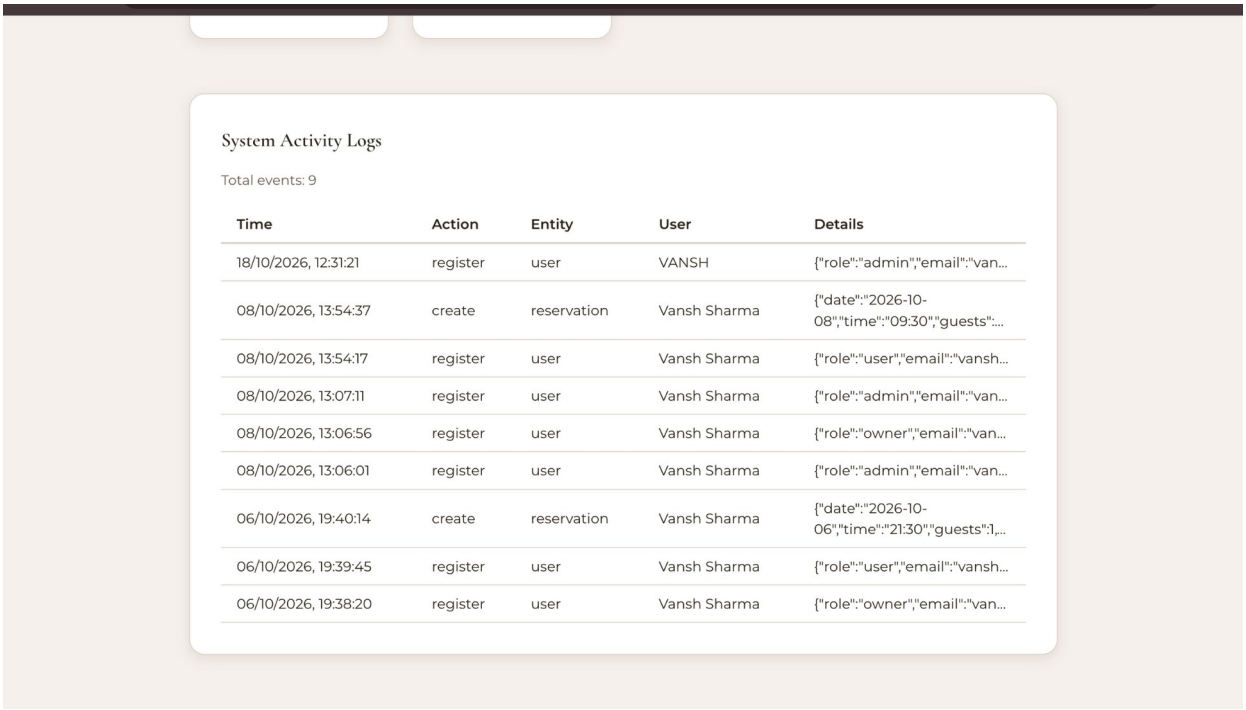


Figure 2. Admin dashboard with user, restaurant, and booking summary cards.

Screenshots and Evidence - Activity Logs and Reservation Flow



System Activity Logs

Total events: 9

Time	Action	Entity	User	Details
18/10/2026, 12:31:21	register	user	VANSH	{"role":"admin","email":"van...
08/10/2026, 13:54:37	create	reservation	Vansh Sharma	{"date":"2026-10-08","time":"09:30","guests":...
08/10/2026, 13:54:17	register	user	Vansh Sharma	{"role":"user","email":"vansh...
08/10/2026, 13:07:11	register	user	Vansh Sharma	{"role":"admin","email":"van...
08/10/2026, 13:06:56	register	user	Vansh Sharma	{"role":"owner","email":"van...
08/10/2026, 13:06:01	register	user	Vansh Sharma	{"role":"admin","email":"van...
06/10/2026, 19:40:14	create	reservation	Vansh Sharma	{"date":"2026-10-06","time":"21:30","guests":1,...
06/10/2026, 19:39:45	register	user	Vansh Sharma	{"role":"user","email":"vansh...
06/10/2026, 19:38:20	register	user	Vansh Sharma	{"role":"owner","email":"van...

Figure 3. System activity logs showing registration and reservation events.

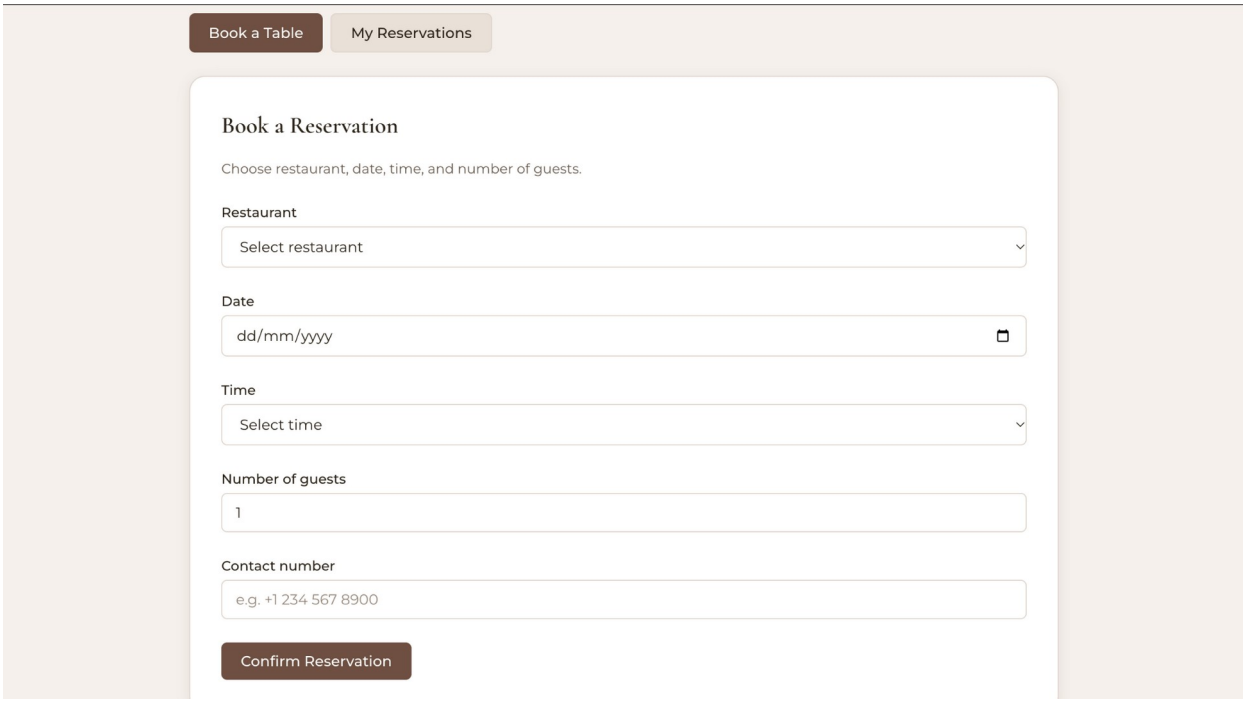


Figure 4. Reservation form for restaurant, date, time, guests, and contact number.

Screenshots and Evidence - Restaurant Search and Jenkins Status

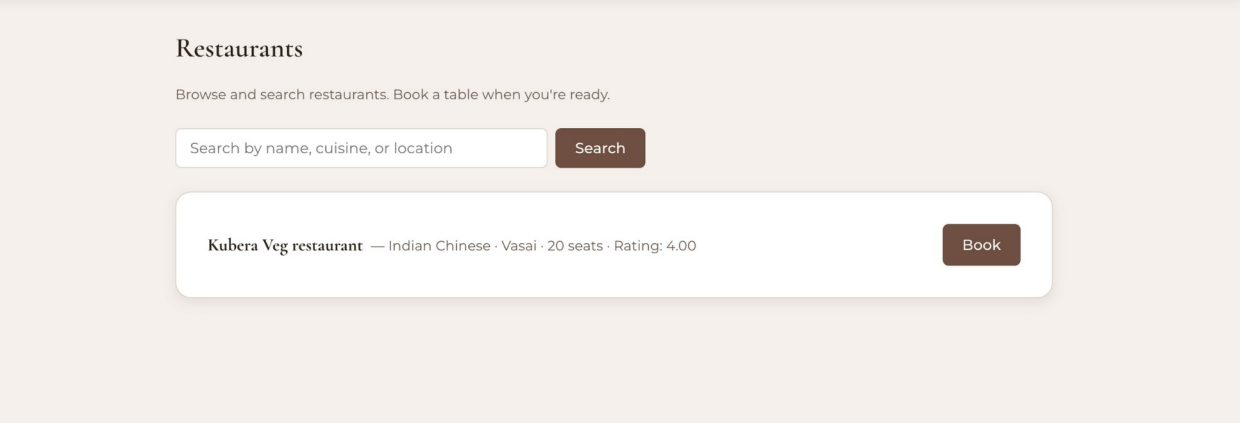


Figure 5. Restaurant discovery and search result for Kubera Veg restaurant.

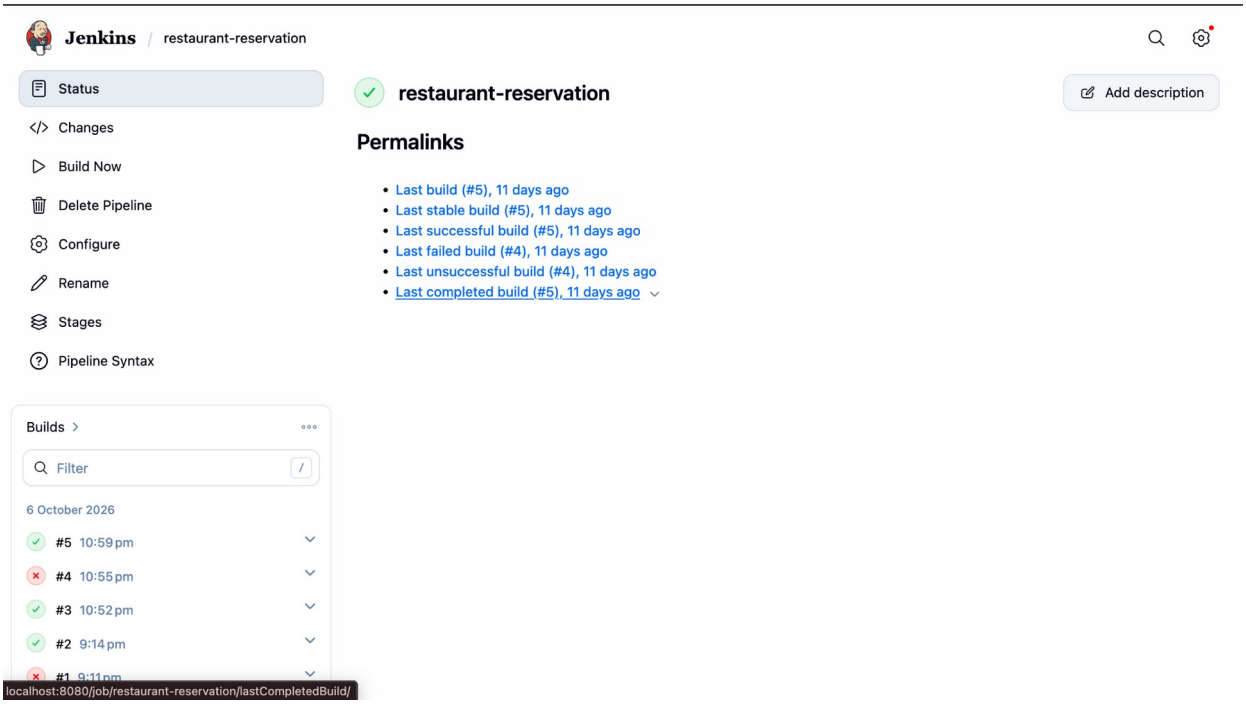


Figure 6. Jenkins pipeline status showing build #5 as successful.

Screenshots and Evidence - Jenkins Changes and Grafana

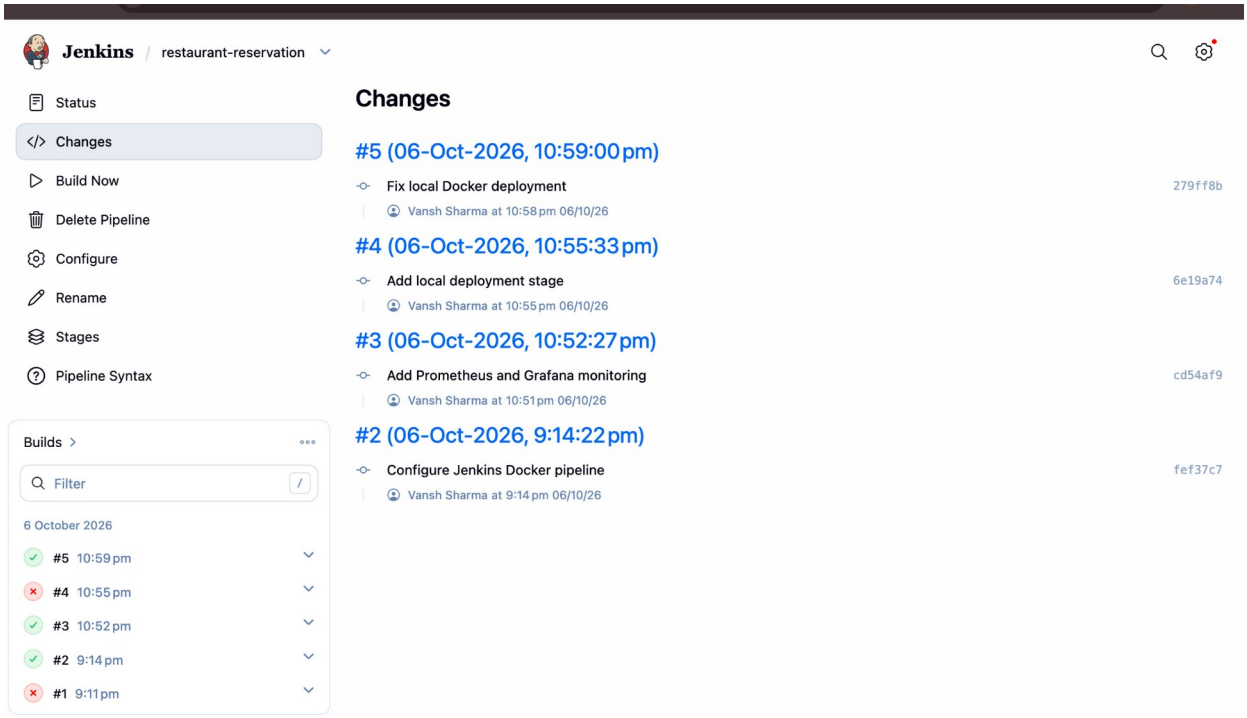


Figure 7. Jenkins change history for pipeline and deployment work.



Figure 8. Grafana resource monitoring dashboard.

Screenshots and Evidence - Jira Sprint and Board

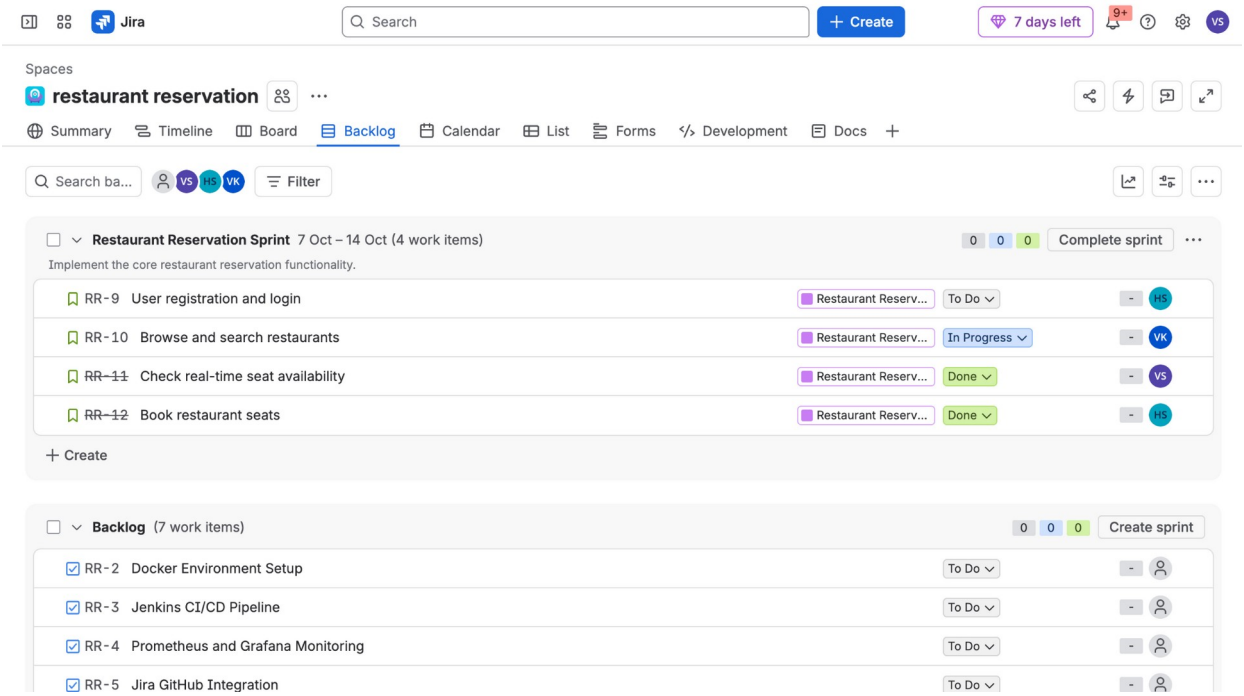


Figure 9. Jira backlog with sprint items and engineering tasks.

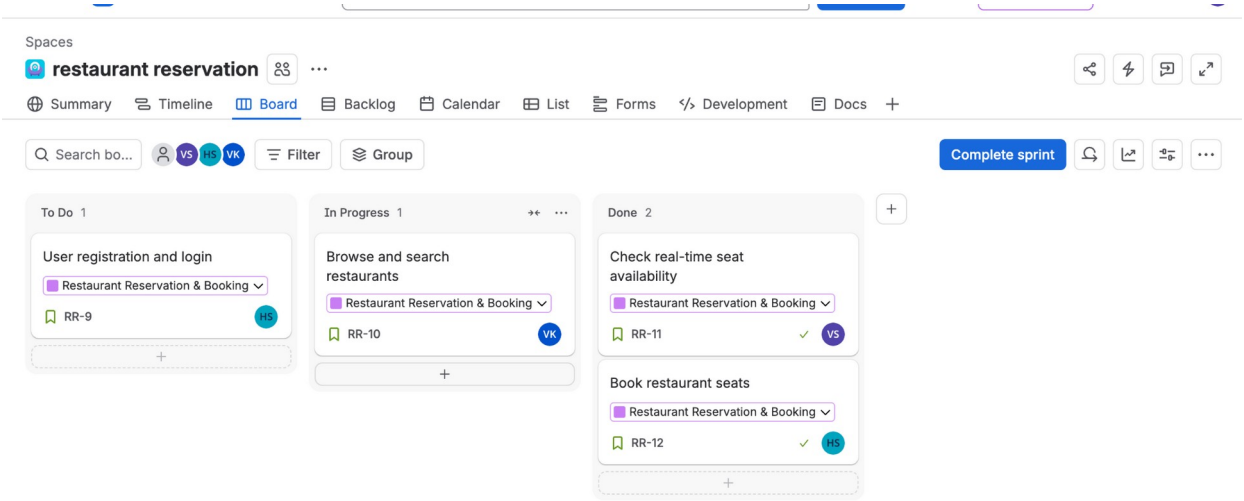


Figure 10. Jira board showing To Do, In Progress, and Done work items.

Screenshots and Evidence - Prometheus and Node Exporter

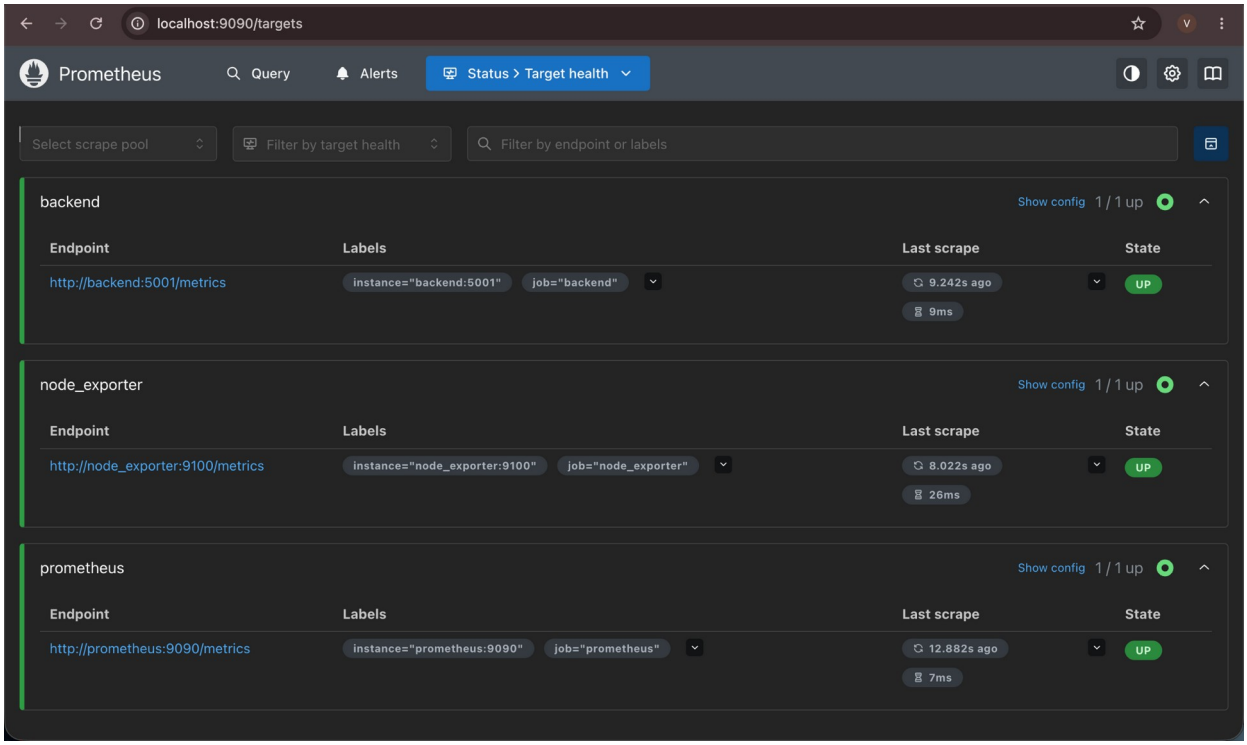


Figure 11. Prometheus target health page showing backend, Node Exporter, and Prometheus targets UP.

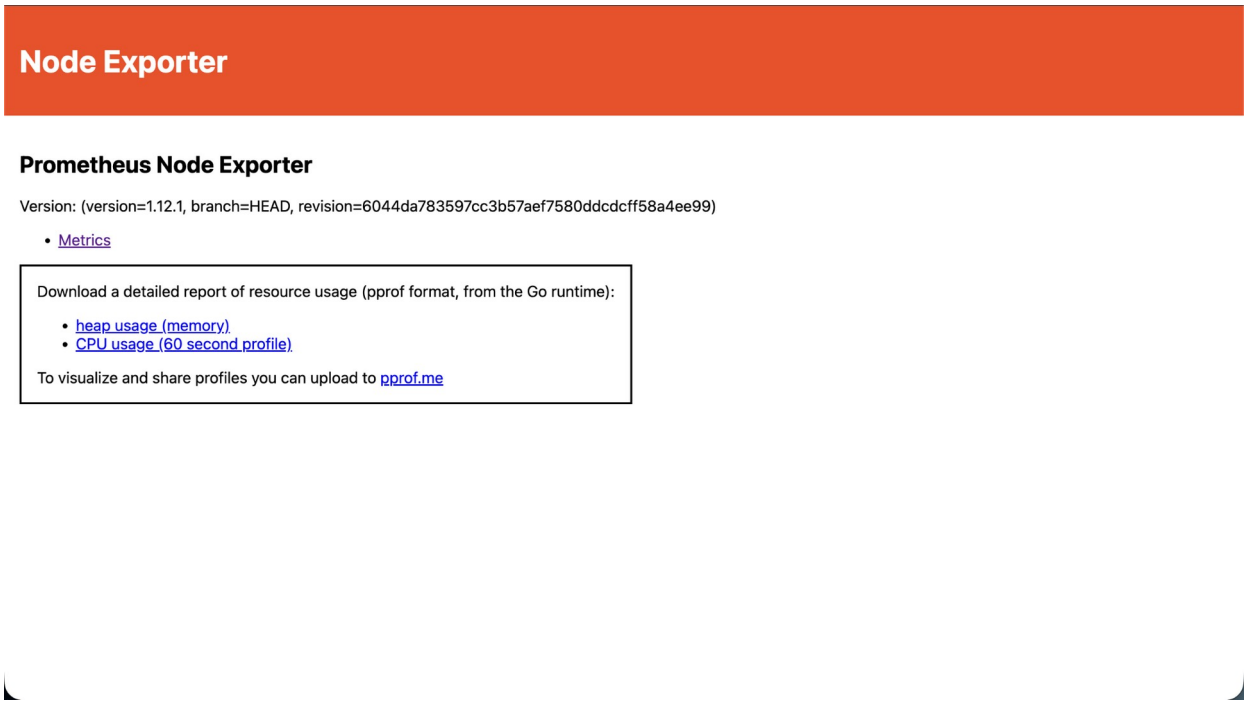


Figure 12. Node Exporter landing page exposing host resource metrics.

Conclusion and Future Scope

Conclusion

The Restaurant Reservation System brings together a restaurant discovery interface, a reservation-entry form, and an administrative view of booking statistics and system events. The supplied screenshots also show supporting project practices through Jira issue tracking, Jenkins build history, and a Prometheus/Grafana monitoring setup with Node Exporter. These parts provide a useful foundation for demonstrating both application functionality and basic engineering operations.

The available evidence supports the presence of the described screens and tools, but a production-readiness conclusion would require repeatable functional, security, integration, and performance testing. In particular, reservation confirmation, persistence, availability conflicts, and authorization should be verified end to end before the system is relied on for live booking operations.

Future Scope

- Add an explicit availability calendar with capacity-aware slot selection and safe handling of simultaneous booking attempts.
- Provide a complete reservation lifecycle: confirmation, modification, cancellation, and a populated My Reservations page.
- Improve admin reporting with filters, date ranges, export, and clearer drill-down into audit events.
- Add automated functional/API tests to the Jenkins pipeline and preserve test reports for each build.
- Review authentication, authorization, secrets management, input validation, and privacy of log details.
- Validate Prometheus metric definitions and Grafana units/labels; add alerts for service unavailability and elevated error/latency rates.
- Run load and recovery tests and document measurements before claiming specific performance or reliability targets.

References

The following official documentation pages provide background on the development and monitoring tools visible in the supplied screenshots. They are included as technical references; the screenshots themselves are the primary evidence used for the implementation descriptions in this report.

1. Jenkins User Documentation. https://www.jenkins.io/doc/

2. Atlassian Jira Software Cloud Support. https://support.atlassian.com/jira-software-cloud/

3. Prometheus Documentation. https://prometheus.io/docs/

4. Prometheus Node Exporter. https://github.com/prometheus/node_exporter

5. Grafana Documentation. https://grafana.com/docs/

6. Docker Compose Documentation. https://docs.docker.com/compose/

Evidence note: Application counts, target states, metric readings, and build statuses are point-in-time values transcribed from the screenshots supplied with this report. They are not generalized benchmarks.